

ASSIGNMENT — 20 QUESTIONS

20
Total Marks

30 min
Time

MCQ
Type

6 — Callbacks &
Event Loop
Day

Instructions

Har question ke liye sabse sahi answer choose karo. Sab questions 1 mark ke hain.

★ questions interview mein frequently aate hain — inka output mentally trace karo.

Answer key aakhiri page pe hai. 90%+ = Day 7 ke liye ready.

Q1. [Sync/Async] JavaScript single-threaded hone ka matlab kya hai?

- (A) Ek se zyada CPU cores use nahi karta
- (B) Ek waqt pe sirf ek operation execute ho sakta hai — event loop se async handle hota hai
- (C) Sirf ek variable store kar sakta hai
- (D) Sirf ek file import kar sakta hai

Q2. [setTimeout] ★ `setTimeout(fn, 0)` ke baare mein kaunsa statement SAHI hai?

- (A) fn turant (synchronously) execute hoti hai
- (B) fn macrotask queue mein jaati hai — stack khaali hone ke baad run hoti hai
- (C) fn microtask queue mein jaati hai
- (D) fn ko cancel nahi kiya ja sakta

Q3. [Event Loop] `console.log('A');` `setTimeout(()=>console.log('B'),0);` `console.log('C');` — output?

- (A) A B C
- (B) A C B
- (C) B A C
- (D) C A B

Q4. [Call Stack] ★ Call Stack kya hai?

- (A) Browser ki memory store karne ki jagah
- (B) LIFO structure jahan function calls track hoti hain during execution
- (C) Async tasks ka queue
- (D) Event listeners ka list

Q5. [Call Stack] Stack Overflow kab hota hai?

- (A) Bahut badi array create karne par
- (B) Recursion bahut deep ho jaye — stack size exceed ho jaye
- (C) setTimeout bahut zyada baar use karne par
- (D) Promise chain bahut lamba ho

Q6. [Event Loop] ★ Microtask Queue aur Callback Queue mein priority kaisi hoti hai?

- (A) Callback Queue pehle drain hoti hai
- (B) Dono same priority pe hain
- (C) Microtask Queue hamesha Callback Queue se pehle drain hoti hai
- (D) Order random hota hai

Q7. [Event Loop] ★ Promise.resolve().then() kaunsi queue mein jaata hai?

- (A) Callback Queue (Macrotask)
- (B) Microtask Queue
- (C) Call Stack
- (D) Web API

Q8. [Event Loop] Event Loop ka kaam kya hai?

- (A) JavaScript code compile karna
- (B) Call Stack aur Queues ko monitor karna — stack khaali hone par queue se task uthana
- (C) Async code run karna
- (D) Memory manage karna

Q9. [Event Loop] ★ Kaunsa output sahi hai? `console.log('1');` `Promise.resolve().then(()=>console.log('2'));` `setTimeout(()=>console.log('3'),0);` `console.log('4');`

- (A) 1 2 3 4
- (B) 1 4 3 2
- (C) 1 4 2 3
- (D) 4 1 2 3

Q10. [Callbacks] Node.js Error-First Callback convention kya hai?

- (A) Error ko last argument mein pass karo
- (B) Callback ka pehla argument hamesha error hota hai (null agar success)
- (C) Errors throw karo, callback nahi
- (D) Errors ignore karo

Q11. [Callback Hell] ★ Callback Hell ke baare mein kaunsa statement GALAT hai?

- (A) Code pyramid/Christmas tree shape le leta hai
- (B) Error handling har level pe repeat karna padta hai
- (C) Testing aur debugging mushkil ho jaati hai
- (D) Performance bahut slow ho jaati hai — main bottleneck hai

Q12. [Callbacks] Error-first callback mein success aane par pehla argument kya hona chahiye?

- (A) undefined
- (B) false
- (C) null
- (D) 0

Q13. [setInterval] ★ setInterval(fn, 100) mein fn ko execute hone mein 150ms lagte hain to kya hoga?

- (A) JS engine automatically wait karega
- (B) Callbacks queue mein stack up honge — overlap ho sakta hai
- (C) setInterval automatically slow ho jaayega
- (D) Error throw hogi

Q14. [setInterval] setInterval ko stop karne ka sahi tarika kya hai?

- (A) setInterval.stop()
- (B) clearInterval(intervalId)
- (C) setInterval.clear()
- (D) delete setInterval

Q15. [setTimeout] ★ Recursive setTimeout vs setInterval — kya fayda hai recursive setTimeout ka?

- (A) Faster execute hota hai
- (B) Previous callback complete hone ke baad hi next timer start hota hai — no overlap, no drift
- (C) Memory efficient hai
- (D) Koi fayda nahi

Q16. [Debounce/Throttle] ★ Debounce kya karta hai?

- (A) Har call pe function immediately execute karta hai
- (B) Function ko delay karta hai aur agar us delay mein naya call aaye to timer reset karta hai — sirf last call ke baad execute hota hai
- (C) Function ko har X ms mein ek baar execute karta hai
- (D) Function ko permanently disable karta hai

Q17. [Debounce/Throttle] Throttle aur Debounce mein main difference?

- (A) Koi farq nahi
- (B) Throttle: maximum frequency limit (X ms mein ek baar). Debounce: silence ke baad ek baar.

- (C) Debounce faster hai
- (D) Throttle sirf click events ke liye hai

Q18. [Debounce/Throttle] Search input field mein user type karte waqt API call minimize karne ke liye best approach?

- (A) Throttle — har keystroke delay karo
- (B) Debounce — typing ruk jaane ke baad (e.g. 400ms) ek API call karo
- (C) setInterval — regular intervals pe call karo
- (D) setTimeout — har keystroke pe 100ms delay

Q19. [Web APIs] setTimeout aur fetch — ye JS execute karta hai ya koi aur?

- (A) JavaScript engine directly execute karta hai
- (B) Browser's Web APIs (ya Node.js libuv) execute karte hain — JS thread free rehta hai
- (C) Operating system execute karta hai
- (D) Call Stack mein wait karte hain

Q20. [Event Loop] ★ Agar ye code chalao: while(true){} — kya hoga?

- (A) setTimeout callbacks run hote rahenge
- (B) Promise.then callbacks run hote rahenge
- (C) Main thread completely block ho jaayega — kuch bhi nahi chalega
- (D) Browser automatically terminate karega

Answer Key

Q1: (B)	Q2: (B)	Q3: (B)	Q4: (B)	Q5: (B)
Q6: (C)	Q7: (B)	Q8: (B)	Q9: (C)	Q10: (B)
Q11: (D)	Q12: (C)	Q13: (B)	Q14: (B)	Q15: (B)
Q16: (B)	Q17: (B)	Q18: (B)	Q19: (B)	Q20: (C)

Scoring Guide

20/20 (100%) — Outstanding! Event Loop concepts crystal clear hain. Day 7 (Promises) ready.

15-19 (75-95%) — Good! Event loop order trace karne wale questions dubara try karo.

10-14 (50-70%) — Theory notes dubara padhein — especially microtask vs macrotask queue.

Below 10 (<50%)— Mentor se mil lo. Event Loop Week 2-8 mein EVERYWHERE aayega.